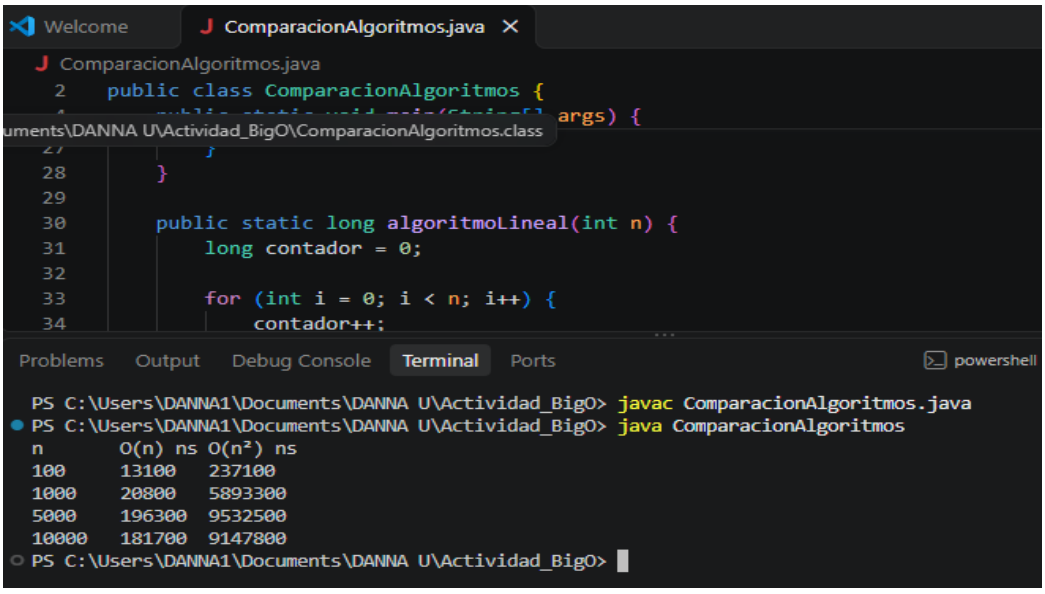
PUNTO 1

Fragmento	Operaciones aproximadas	Big O	Justificación
Acceder a vector[5]	1	O(1)	Accede directamente a una posición del arreglo.
Recorrer un arreglo	n	O(n)	Recorre todos los elementos una sola vez.
Dos ciclos anidados	$n \times n$	$O(n^2)$	Un ciclo está dentro de otro y aumenta las operaciones.
Dividir n entre 2 repetidamente	$\log_2(n)$	O(log n)	En cada repetición el valor se reduce a la mitad.
Ciclo n + reducción logarítmica	$n \times \log_2(n)$	O(n log n)	Combina un ciclo que se repite n veces con otro que reduce el valor a la mitad.

PUNTO 2

- **Algoritmo A – O(n):** realiza una operación por cada elemento.
- **Algoritmo B – O(n²):** utiliza dos ciclos anidados, por lo que realiza muchas más operaciones cuando aumenta la cantidad de datos.

PUNTO 3



PUNTO 4

TAMAÑO N	TIEMPO O(N)	TIEMPO O(N^2)
100	13.100 ns	237.100 ns
1.000	20.800 ns	5.893.300 ns
5.000	196.300 ns	9.532.500 ns
10.000	181.700 ns	9.147.800 ns

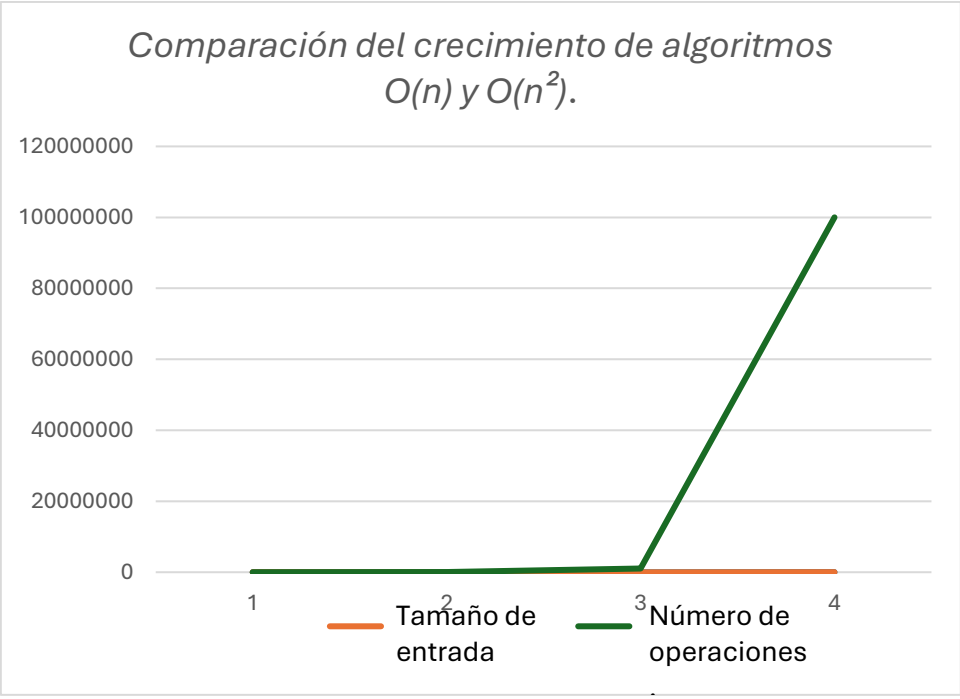
Los resultados muestran que el algoritmo lineal O(n) fue más rápido que el algoritmo cuadrático O(n^2) en todas las pruebas. Esto demuestra que el algoritmo cuadrático necesita mucho más tiempo cuando aumenta la cantidad de datos. Aunque los tiempos pueden variar entre ejecuciones, se observa que O(n) presenta un mejor rendimiento.

PUNTO 5

n	O(1)	O(log n) aprox.	O(n)	O(n log n) aprox.	O(n^2)
10	1	3,32	10	33	100
100	1	6,64	100	664	10.000
1.000	1	9,97	1.000	9.966	1.000.000
10.000	1	13,29	10.000	132.877	100.000.000

PUNTO 6

n	O(n)	O(n^2)
10	10	100
100	100	10.000
1.000	1.000	1.000.000
10.000	10.000	100.000.000



PUNTO 7

1. ¿QUÉ DIFERENCIA EXISTE ENTRE TIEMPO DE EJECUCIÓN Y COMPLEJIDAD BIG O?

- El tiempo de ejecución indica cuánto tarda un programa en realizar una tarea, mientras que Big O muestra cómo aumenta el trabajo del algoritmo cuando crecen los datos.

2. ¿POR QUÉ O(1) SE CONSIDERA UNA COMPLEJIDAD EFICIENTE?

- Porque realiza una cantidad constante de operaciones, sin importar cuántos datos tenga el arreglo.

3. ¿QUÉ OCURRE CON UN ALGORITMO O(N) CUANDO DUPLICAMOS EL TAMAÑO DE ENTRADA?

- El número de operaciones también se duplica, porque su crecimiento es proporcional a la cantidad de datos.

4. ¿QUÉ OCURRE APROXIMADAMENTE CON UN ALGORITMO O(N^2) CUANDO DUPLICAMOS N?

- El número de operaciones se multiplica por cuatro, porque depende de n al cuadrado.

5. ¿POR QUÉ LOS CICLOS ANIDADOS SUELEN RELACIONARSE CON O(N^2)?

- Porque un ciclo se encuentra dentro de otro y, por cada repetición del primero, se ejecuta nuevamente el segundo.

6. ¿QUÉ CARACTERÍSTICA PERMITE RECONOCER UN ALGORITMO O(LOG N)?

- Que en cada repetición reduce el tamaño del problema, generalmente dividiéndolo entre dos.

7. ¿CUÁL DE LOS DOS ALGORITMOS EXPERIMENTALES PRESENTÓ MAYOR CRECIMIENTO?

- El algoritmo cuadrático $O(n^2)$, porque realiza muchas más operaciones a medida que aumenta la cantidad de datos.

8. ¿LOS RESULTADOS MEDIDOS FUERON EXACTAMENTE IGUALES ENTRE LAS DIFERENTES EJECUCIONES? ¿POR QUÉ?

- No necesariamente, porque los tiempos dependen del procesador, la memoria, el sistema operativo y otros procesos que estén funcionando en el computador.

9. SI TUVIERA QUE PROCESAR DIEZ MILLONES DE REGISTROS, ¿QUÉ TIPO DE COMPLEJIDAD PREFERIRÍA?

- Preferiría un algoritmo con complejidad O(n) o menor, porque permite procesar grandes cantidades de información de manera más eficiente.

10. ¿POR QUÉ UN INGENIERO DE SOFTWARE DEBE ANALIZAR LA EFICIENCIA ANTES DE SELECCIONAR UN ALGORITMO?

- Porque debe buscar que los programas funcionen correctamente y también que no consuman demasiados recursos ni tarden mucho cuando aumenten los datos.

PUNTO 8

¿Los dos ciclos consecutivos representan $O(n^2)$?

RTA: No es $O(n^2)$, porque los dos ciclos están separados y no uno dentro del otro. Cada ciclo se ejecuta n veces, por lo tanto, en total se realizan 2n operaciones. Al simplificar, la complejidad es O(n).

CONCLUSIÓN

Con esta actividad aprendí que la notación Big O permite identificar qué tan eficiente puede ser un algoritmo dependiendo de la cantidad de datos que debe procesar. También comprendí que no todos los algoritmos trabajan de la misma manera, ya que algunos realizan pocas operaciones y otros aumentan considerablemente su trabajo. Al comparar los algoritmos lineal y cuadrático pude entender por qué es importante seleccionar algoritmos eficientes, especialmente cuando se trabaja con grandes cantidades de información. Como futura ingeniera de software, considero importante analizar el rendimiento de los programas para desarrollar aplicaciones que funcionen correctamente y aprovechen mejor los recursos del computador.